DesignKarlo — Research Note: Deliberate LLD Practice Through Iterative Feedback

Document Type: Foundational Product & Learning Research Note
System: DesignKarlo — Low-Level Design Practice & Progression Platform
Target Milestone: MVP Deliverable
Repository: https://github.com/shrajal01/DesignKarlo
Live Application: https://designkarlo.netlify.app/

1. Problem Statement

Low-Level Design (LLD) and Object-Oriented Design (OOD) form a critical component of technical software engineering interviews and real-world system development. While High-Level Design (HLD) emphasizes macro-architectures (caching, database sharding, message brokers), LLD evaluates an engineer's ability to model domain entities, enforce clean boundaries, adhere to SOLID principles, select appropriate design patterns, and anticipate concurrency edge cases.

Despite its importance, preparing for LLD remains one of the most fragmented and frustrating phases of technical preparation. Candidates lack standardized environments to practice modeling, test architectural trade-offs, and receive actionable, objective critique on their designs.

2. Target Learners and Their Pain Points

Target Learners

- Software engineers and computer science students preparing for technical LLD and OOD interviews.
- Developers seeking to transition from writing procedural scripts to designing modular, extensible object-oriented architectures.
- Technical mentors and interviewers seeking a structured rubric to evaluate candidate architectural maturity.

Core Learner Pain Points

1. **The Ambiguous Feedback Loop:** When candidates practice classic problems (e.g., *Parking Lot*, *Elevator System*), they typically sketch on paper or read static solutions. There is no feedback loop to evaluate whether their class boundaries are cohesive, interfaces are properly segregated, or coupling risks exist.
2. **The Illusion of Competence:** Reading or watching solutions to 20 different problems creates passive recognition without building design intuition. Architectural maturity is forged when a learner makes a design mistake, receives concrete critique, refactors the model, and observes tangible improvement.
3. **Drawing Overhead vs. Architectural Reasoning:** Generic diagramming canvases require candidates to spend excessive time manipulating shapes, resizing boxes, and routing arrows rather than reasoning about responsibilities, encapsulation, and contracts.

3. Existing Approaches and Their Limitations

Approach	Typical Tools	Characteristics & Observed Trade-offs
Visual Diagramming & Canvas Tools	Draw.io, Lucidchart, PlantUML, LLDCanvas	Valuable for visual architecture and layout design. LLDCanvas exists as a dedicated canvas-first platform serving interactive visual diagramming. However, canvas-centric tools focus on visual composition and spatial layout rather than automated rubric-based grading, objective debt critique, or structured revision diffing.
Curated GitHub Repositories	Public LLD interview repositories	Popular for self-study and community reference solutions. However, as an observed product friction, the learning experience can be fragmented across separate problem READMEs, disjointed code files, isolated pattern examples, and detached discussions, without an integrated draft-critique-revise workflow.
Algorithmic Platforms	LeetCode, HackerRank	Highly effective for testing data structures and algorithmic complexity, but evaluate binary test outputs rather than object-oriented modularity, encapsulation, or extensibility.
Static Text & Video Materials	Technical blogs, YouTube walkthroughs	Useful for passive reference, but candidates do not practice active trade-off reconciliation or receive objective feedback on their own formulations.
Generic AI Chatbots	ChatGPT, Claude prompts	Helpful for qualitative summaries, but typically generate one-off, non-deterministic responses without persistent rubric anchoring, multi-attempt continuity, or quantifiable revision comparisons.

4. Product Hypothesis and DesignKarlo's Proposed Value

Core Product Hypothesis

> **If** candidates author object-oriented designs within a structured, schema-driven environment and receive immediate, principle-anchored rubric feedback with multi-revision comparisons,
> **Then** they will resolve architectural smells faster and demonstrate measurable improvements in modularity compared to passive reading or unguided diagramming.

Proposed Value Proposition

1. **Structured Submissions:** A schema-driven Form and JSON editor modeling Assumptions, Classes, Relationships, Patterns, and Concurrency, reducing visual canvas overhead.

2. **Consistent Rubric-Based Feedback (`deterministic-v1`):** A 100-point rule-based engine scoring four 25-point dimensions (Domain Entities, SOLID Modularity, Extensibility, Edge Cases) with 100% reproducibility and zero external API latency.
3. **Actionable, Evidence-Backed Feedback:** Categorizes feedback into `strength`, `concern`, and `suggestion`, linking issues directly to declared entities.
4. **Attempt History & Revision-to-Revision Comparison:** Maintains multi-revision history per attempt, computing score deltas (`+/-` points) and explicitly identifying resolved, unresolved, and newly introduced architectural concerns.

---

5. MVP Scope and Rationale

The MVP focuses deliberately on a single, comprehensive benchmark problem: **The Parking Lot System**.

- **Pedagogical Richness:** The problem encapsulates core OOP fundamentals: polymorphic vehicle types, spot allocation policies (Strategy pattern), pricing algorithms, ticket lifecycles, and capacity/concurrency edge cases.
- **Dual Practice Entry Flows:** Implements both a **Guided Practice Session** (instructional Problem Briefing detailing functional requirements, assumptions, and edge cases before designing) and a direct **Start Attempt** flow for rapid drafting.
- **Reference Baselines:** Provides distinct **Load Starter** (minimal 2-class baseline) and **Load Sample Architecture** (complete reference implementation) workflows protected by a safe confirmation modal.
- **Zero-Downtime Dual Persistence:** Uses MongoDB Atlas for production storage with an automatic in-memory fallback, ensuring the platform runs immediately in local reviewer environments.

---

6. Hypotheses, Evidence, and Validation Gaps

To maintain strict research integrity, observations, working hypotheses, and empirically verified findings are clearly distinguished:

Statement	Type	Status	Evidence & Validation Notes
Engineers struggle to find actionable, principle-anchored LLD feedback	Product Observation	**Widely Reported**	Documented consistently across developer forums and interview debriefs. Formal survey validation planned.
Studying LLD via GitHub repos feels fragmented across READMEs and code files	Product Observation / Hypothesis	**Hypothesis (Pending Validation)**	Observed learner pattern; requires user interviews to assess workflow friction across multi-file repositories.
Structured text/form entry enables faster authoring than drag-and-drop UML	Working Hypothesis	**Pending Usability Testing**	Requires empirical time-to-first-draft benchmarking comparing structured form input against canvas tools.
Showing side-by-side concern diffs accelerates refactoring across revisions	Working Hypothesis	**Implemented (Prototype)**	Requires longitudinal cohort tracking to measure whether resolved concerns stay resolved in subsequent problems.
Rule-based deterministic heuristics can reliably score structural LLD constraints	Finding	**Verified in Implementation**	Automated test suites (50 API tests, 37 Web tests) confirm reliable detection of god classes, missing interfaces, and pattern targets.

---

7. Success Signals and Next Validation Steps

Success Signals (MVP Metrics)

- **Revision Engagement:** Percentage of user attempts that progress to Revision #2 or beyond (targeting $\ge 40%$).
- **Concern Resolution Rate:** Average number of architectural concerns flagged in Revision #1 that are resolved in Revision #2.
- **Validation Completion:** Ratio of initiated drafts that successfully pass Zod schema validation and receive an evaluation score.

Next Validation Steps

1. **User Usability Pilot:** Conduct supervised testing with 10–15 candidate engineers to evaluate authoring speed and clarity in Form vs. JSON mode.
2. **Rubric Calibration:** Benchmark deterministic scores against senior engineering interview evaluations to fine-tune rubric deductions.
3. **Catalog Expansion:** Register additional classic problems (*Rate Limiter*, *Elevator System*) upon validation of the core progression loop.